



Section 8 / 9

GraphQL Vulnerability Prevention

After discussing how to attack different vulnerabilities that arise from misconfigured GraphQL implementations, let's discuss mitigations to prevent these vulnerabilities.

Vulnerability Prevention

Information Disclosure

General security best practices apply to prevent information disclosure vulnerabilities. These include preventing verbose error messages and displaying generic error messages instead. Furthermore, introspection queries are potent tools for obtaining information. As such, they should be disabled if possible. At the very least, whether any sensitive information is disclosed in introspection queries should be checked. If this is the case, all sensitive information needs to be removed.

Injection Attacks

Proper input validation checks need to be implemented to prevent any injection-type attacks such as SQL injection, command injection, or XSS. Any data the user supplies should be treated as untrusted until it has been appropriately sanitized. The use of allowlists should be preferred over denylists.

Denial-of-Service (DoS)

As discussed, DoS attacks and the amplification of brute-force attacks through batching are common GraphQL attack vectors. Proper limits need to be implemented to mitigate these types of attacks. This can include limits on the GraphQL query depth or maximum GraphQL query size, as well as rate limits on the

GraphQL endpoint to prevent multiple subsequent queries in quick succession. Additionally, batching should be disabled in GraphQL queries whenever possible. If batching is required, the query depth needs to be limited.

API Design

General API security best practices should be followed to prevent further attacks, such as attacks against improper access control (for instance, IDOR) or attacks resulting from improper authorization checks on mutations. These best practices include strict access control measures based on the principle of least privilege. In particular, the GraphQL endpoint should only be accessible after successful authentication, if possible, in accordance with the API's use case. Furthermore, authorization checks must be implemented to prevent actors from executing queries or mutations to which they are not authorized.

For more details on securing GraphQL APIs, check out OWASP's [GraphQL Cheat Sheet](#).



8 / 9 Sections



Mark Complete & Next